A Design and Implementation of Spatial Database Based on XML-SVG

Keywords: **XML-SVG, spatial database, spatial information system**

Jianting Wen,

Computer Science Department, South China Normal University
Guangzhou
GuangDong
China
wenjianting@hotmail.com

Biography

Yan Li, Prof.
Director
Spatial Information Research Center, South China Normal University
Guangzhou
GuangDong
China

Biography

Abstract

SVG, which stands for Scalable Vector Graphics, is an open standard in text that makes it possible to represent graphic information in compact, portable form and mostly faces on publishing the spatial vector graphics in network environment. In practice, it's impossible to avoid a great capacity of spatial data processing, which had made SVG document is too big to be implemented during publishing on the Internet. It is slowed down the executing speed of the system on client side and is weaken its advantages and superiority. Such SVG database has been proposed as the solution for managing all the graphics and its attributes in a Web server. SVG document is mapped into SVG database that works as a buffer in the server. The issuance efficiency has been improved to a great extent. In order to prove the result, this paper has been taken a case study of the spatial information service system of Guangdong Province.

Table of Contents

1. Introduction
2. The characteristic and data structure of spatial SVG data
3. Design of graphic database based on SVG
3.1 table-based mapping
3.2 Object-relational based mapping
4. Implementation of SVG database
4.1 the system framework
4.2 data source and the configuration of relational database
4.3 storage of SVG data into SVG database
4.4 issuance and alternation
5. Conclusion and prospect
Bibliography

1. Introduction

In general, there are two types of computer graphics formats - raster and vector. The raster graphic is composed of pixels such as a bitmap is basically an array of pixels with the color or grey, brightness and attributes defined in an umpteen binary. The most graphics formats used in the Internet are raster graphics like BMP, TIF, GIF, JPG and PNG. Since the bitmap sizes are defined in pixels, they are always very big file even for a small image. And a vector graphic is composed of drawing points, lines and polygons or paths as objects. Each object can be connected to its attributes to describe a position, color and other related information. The size of a vector graphic is much smaller than raster graphic. And it can be easily scaled without losing resolution and features on the graphics.

SVG (Scalable Vector Graphics) format is a new vector graphics standard based on XML grammar for defining vector-based 2D graphics on Web. SVG has more advantages than the most common graphic formats used on Web today. Specifically, it is a high-resolution graphics with plain text format which is scalable, searchable text, filtering, animation and scripting etc abilities. The most advantage of SVG is the Open standard of True XML. It offers all the advantages of XML, and it can be easily manipulated through standard APIs such as the Document Object Model (DOM) API and SAX. It can also be easily transformed through XML Style sheet Language Transformation (XSLT) that also utilizes DOM.

Currently, the spatial information system has realized advanced web mapping with SVG , such as map navigation, interactive map operation, layer control and attribute query. However, a great capacity of spatial data issuance has greatly affected the interactive speed on client side. For example, when there is a request from SVG client, the server always sends the whole SVG document to response to the client. In fact, it is not necessary to send all the information for the user but what the client interested in. It means that all the information even if graphic information stored in the same document is not a good way to manipulate spatial data on client side. This paper was just taken this issue to discuss the approach of SVG database creation and the related solutions to catabolize the speed of the SVG document publishing on Web.

2. The characteristic and data structure of spatial SVG data

Due to SVG is defined as a graphics standard publishing the vector graphic and the images on Internet, it was laid out many achievements in the graphics applications. However the spatial data are rather more complex than that and a spatial SVG document is too large to be well organized for an efficient browsing and managing, especially, a spatial information project. Hence, a solution of organizing and storing SVG based spatial data has to be found in creating spatial databases. This should be started with two aspects: one is the classification of the geographic entities; another is the organization of geographic entity information.

Many researching works had discussed the classification of geographic entities expression **[1] [2]** . The common solution is to describe them as points, lines, polygon, complex polygon, raster and annotation. These features will be enough to express the geographic entities in real world.

Point: it is such an entity that has specified position but no extent, such as level point, highway mere stone etc.

Line: it is such an entity that has extent, such as road, canal and river etc.

Polygon: it is such an entity that has a certain area, such as lake, district region and flied etc.

Annotation: it is a suppositional entity; it is used to describe the three types of the entities described above in characters.

Raster: it uses raster images to express geographic phenomenon.

According to the basic principles, some basic graphic elements of the geographic entities were defined to express them in SVG document, such as ellipse or circle element used to express point, path element to express a line and a group of paths to express a polygon, text element to express annotation and image element to express raster.

There are two spatial data organizing approaches: the layers organizing and elements organizing. The former one comes from special layers in cartography or classifications and the entities layers in CAD. Its basic principle is to divide the spatial data into several layers according to different types, such as road layer, building layer etc. the layers are independent from each other, they can be overlapped for analyzing model and making decision. The later one is defined the elements as the basic expression unit, similar elements can be grouped together, and even different element can be composed a complex element. Because of the complex data structure, it is preferred a support of object oriented database but no commercial OODBMS available yet. Then, all the elements are classified by basic elements and layered to fit the layers organization in RDBMS.

In order to organize the spatial data in SVG format, the features of SVG expression or data structure for graphics should be studied and then classified into different layers based on the basic elements such as point layer, line layer, polygon layer, annotation layer and raster layer. A layer is expressed with 'g' as a grouped elements, and element ID is the identifier of the layer or group of elements in SVG document. Different layer has its own displaying stylesheet.

1. Point layer:For the point layer, it is expressed by point denotation. Not only its color but also its symbol should be defined. Here is give an example below.
   ```
   <ellipse cx="avg_X" cy="avg_Y" rx="pt-size" ry="pt-size" stroke="pt-color" fill="pt-color"
   UserID="pt-UserID" SysID="pt-SysID" attribute1 attribute2...>
   ```
   In this example, SysID is the system identifier, UserID is the identifier defined by user etc. they are all provided by different data sources such as ArcInfo, MapInfo, or Oracle. Although there is no definition in the SVG criterion, all of them can be defined for the demands. SVG plug in script may ignore them but they can be processed in the other programs.
2. Line layer:In SVG format, path is a expression of a line instead of the conversional expression to the line symbol, for example
   ```
   <path UserID="Arc-UserID" SysID="Arc-SysID" other attributes d="M ptsx[0], ptsy[0]
   ptsx[1],ptsy[1]..." />
   ```
   The simple line style are defined using stroke, stroke-with etc for the whole layer in the <g> marker.
3. Polygon layer:In SVG format, the path element is used to describe polygon, for example

```
<path SysID="" UserID="" Area="" Perimeter="" Other attribute groups d="..."/>
```

And its stylesheet definition is similar with the line layer.

4. The annotation layer style is comparatively simple, a style attribute is just defined as 'stroke'.
5. The raster layer has no style definition to be considered.

The different types of the entities described above can be organized in the SVG document shown in Table 1.

Table 1 description of SVG document

Xml version etc.
Document type description in SVG and some other extra attribute description
SVG element etc. define viewBox and coordinate transform
Point layer (choose)
Line layer (choose)
Polygon layer (choose)
Annotation layer (choose)
Raster layer (choose)

Table 1

A SVG template was given in below

```
<?xml version="1.0"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 20010904//EN"
  "http://www.w3.org/TR/2001/REC-SVG-20010904/DTD/svg10.dtd"
  [<!ATTLIST path attribute1 attribute 2 attribute 3...>
  <!ATTLIST ellipse attribute 1 attribute 2 attribute 3...>
  ...]>
   <svg viewBox="x y Width Height">
     <g fill="none" stroke="black" stroke-width="0.5 " transform=" coordinate transform ">
       <g id="Point Layer"...>
          <ellipse ...>
          <ellipse ...>
          ...
        </g>
     </g>
   </svg>
   <svg viewBox="x y Width Height">
   <g fill="none" stroke="black" stroke-width="0.6" transform=" coordinate transform ">
     <g id="Line Layer"...>
       <path...>
       <path...>
       ...
     </g>
   </g>
   </svg>
   <svg viewBox="x y Width Height">
   <g fill="none" stroke="black" stroke-width="0.5" transform=" coordinate transform ">
     <g id="Polygon_Layer"...>
       <path ...>
       <path ...>
      ...
     </g>
    </g>
```

```
</svg>
```

The attribute query is an important function in the spatial information system. There are always many attribute data related to the geometric graphic entities. In generally, two approaches are used to organize and to store the attribute data in the SVG document. One is the outside connecting approach which is stored the attribute data and the graphic data separately. The graphic data is stored in the SVG document. And the attribute data be stored in RDBMS in sever. Both of them connected with the unified id or key field. Another inserted approach is grouped and stored the attribute data with graphic data together in the SVG document. And it is also contained in some extra attribute that is defined by the users.

In many cases, the former three types of the entities are used in outside connecting approach but the annotations are stored as the attributes and the raster format images are stored in image base.

3. Design of graphic database based on SVG

Since the SVG document is an xml based plain text file, how to manage this kind of XML Marked data and if it can be stored and managed in the DBMS for querying and operating. The former studies shown **[3] [4] [5]** , there are two types of XML databases that XML-enabled database is not stored the data as XML format in XML database; and native XML database is stored the data as XML format in XML database. The former type can just be stored in one of the commercial DBMS whether it is a relational or Object oriented DBMS. It is not very difficult to map the SVG elements and attributes of the document to relational database, which was chosen to store the SVG data.

There are three choices to store SVG data in RDBMS **[3]** . The first one is the fine granularity method, which every element, attribute and text can be accessed, updated or deleted individually as each of them has its own identifier. The advantage of this method is easy to query and update. And the disadvantage of it is the expensive in storage and the restoration. The next one is called a coarse granularity method to store the whole document as a record. This method is not easy to operate with every element but easy to do it with whole document. The third one is called a medium granularity method, which was divided the whole document into several segments and stored them individually as a record. The creation of dividing points is according to the indexing structure of the database or physics memory as a buffer mechanism. In this method, the dividing point can be assigned to a <g> in order to transfer the layers easily. However, users often have to manipulate with each element and attribute, thus, the fine granularity method was chosen for the case study. The two mapping methods are provided: the table-based mapping and object-relational based mapping **[4] [5]** .

In the mapping procedure, spatial SVG data format is important to be studied for mapping but not to the whole SVG DTD.

3.1 table-based mapping

Similar with the xml, there is an obvious mapping relationship between the SVG document and the tables shown in following example in table 2.

```
<g>
 <path sysid="aaa" userid="bbb" d="M 0 1 2 3 Z"/>
 <path sysid="ccc" userid="fff" d="M 5 6 7 8 Z"/>
</g>
```

Table 2 table content of SVG document

Table g

Table g		
sysid	userid	d
aaa	bbb	M 0 1 2 3 Z
ccc	fff	M 5 6 7 8 Z

Table 2

It was clear that a document as a single table or a set of tables. Each column of data can represent attributes. But the next method can express a more natural structure than this one, it is described in detail next section.

3.2 Object-relational based mapping

The object-relational based mapping is regarded the data in XML document as a specific model of object tree. In this model, elements and their types are regarded to be the classes such as attributes, element content, or mixed content (complex element types). And all the element types only with PCDATA content (simple element types), attributes, and PCDATA are assigned to be simple attributes. This means that the classes were mapped into the tables but simple attributes were mapped into the fields and the attributes of the objects were assigned as the primary key or the foreign key. It is important to understand that the object model used in the mapping is not the Document Object Model (DOM). Here were give an example to explain how to map the SVG document into the tables of relational DBMS in following codes **Figure 1** and **Figure 2** .

```
<svg viewBox="0 0 5740 3545" id="mainview">
    <g id="Coordinary transform" transform="translate(0,3545 ) scale(1,-1)"
fill="none" stroke="black" stroke-width="0.05803125">
      <g id="zq_net20 ">
        <path AREA="46.25" PERIMETER="42" SysID="2" UserID="54808"
onclick="show(2)" d="M 5388.5 3545 l 0 –2.5 –18.5 0 0 1.5 0 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1.5 0 Z" />
        <path AREA="3.75" PERIMETER="8" SysID="3" UserID="54809"
onclick="show(3)" d="M 5388.5 3542 l 0 2.5 1.5 0 0 –1 0 –1.5 –1.5 0 Z" />
      </g>
    </g>
</svg>
```

Figure 1: main structure of the document

In the first part, complex element types are modeled as classes. The second part is the object-relational mapping, which classes are mapped to tables (known as class tables), scalar properties are mapped to columns, and pointer/reference properties are mapped to primary key/foreign key relationships. Because the relationship between the parent and child elements is one-to-one, the primary key can be in either table. If the relationship is one-to-many, the primary key must be on

the "one" side of the relationship, regardless of whether it is the parent or the child.

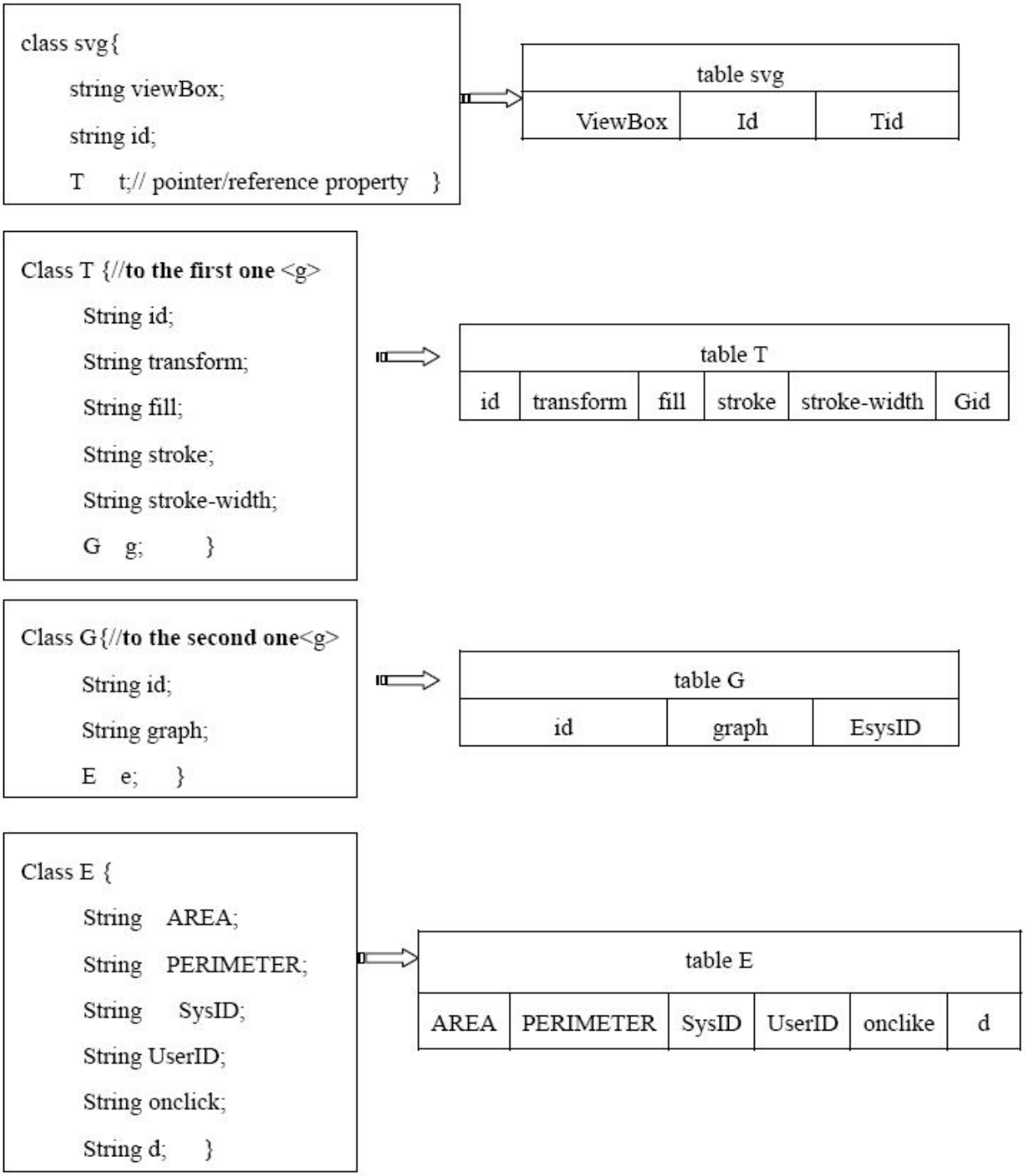


Figure 2: Mapping relationship between SVG document and relational DBMS

Tid , Gid, EsysID are foreign keys. Considering that tables between svg and T, and table T and G are one-to-one, the first three tables can be combined into one in **Figure 3**

table SVGTG									
viewBox	svgid	Tid	transform	fill	stroke	stroke-width	Gid	graph	EsysID

Figure 3: a combined table for Fig.2

In order to record the related information of the SVG documents, a document table is needed to be created such as Document (id, name, info) and info is the information on top part. Because of the heritability, the version and SVG document types were not recorded. What was information recorded just those extra attribute definitions. At the same time, a foreign key was appended to the document id as in table SVGTG. Finally, an index was created for these frequently accessed fields according to the demands. For example, the indexes of administrative districts and international cartologic standards were created after connecting the elements in each layer between them. This is provided a pyramid query approach to access more detail information gradually without the pressure or the limitation of network transmission and the configure of hardware.

4. Implementation of SVG database

Based on the design above, all the data can be stored in database. Let's explain it with an example below.

4.1 the system framework

The database structure of implementation is shown in **Figure 4**

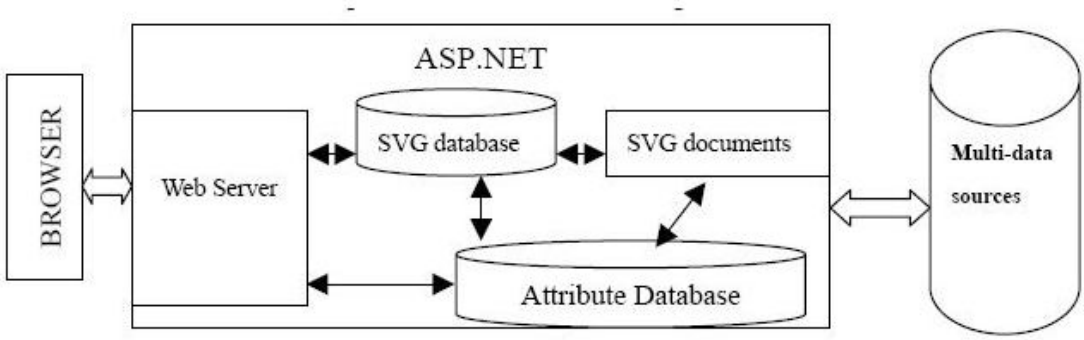


Figure 4: Architecture of Implementing SVG Database

In fact, SVG database and attribute database are bound together.

4.2 data source and the configuration of relational database

The SVG document comes from ArcInfo e00 formatted data; its configuration is just like what has said above. There are four layers: **Figure 5**

Administrated district layer:

<svg viewBox=" 0 0 889986.770735918 616008.63322442">

<g id="Coordinary transform" transform="translate(0,616008.63322442) scale(1,-1)" fill="mintcream" stroke="black" stroke-width="941.247127475211">

<g id="bndcodeok2.pat">

<path d="M 426182 589176 l -111 -306 -77 -108 -79 -42 -97 -11 -279 -14 -291 -32 -97 -29 -120 ……17 -315 -58 -467 -132 -613 -149 -556 20 -250 3 -15 Z" />;

Double river layer:

<svg id="DBriver" viewBox=" 0 0 889986.770735918 616008.63322442">

<g id="Coordinary transform" transform="translate(0,616008.63322442) scale(1,-1)" fill="mintcream" stroke="black" stroke-width="941.247127475211">

<g id="dbriv_alb_ok.aat">

<path d="M 534855.703125 585799.382553926 l 126 297 -32 330 -290 452 463 -169 -191 632 ……-309 41 -838 887 271 908 526 728 598 552 246 " />。

Single line river layer:

<svg id="sglRiver" viewBox=" 0 0 889986.770735918 616008.63322442">

<g id="Coordinary transform" transform="translate(0,616008.63322442) scale(1,-1)" fill="mintcream" stroke="black" stroke-width="941.247127475211">

<g id="sglriv_alb_ok.aat">

<path d="M 401389.533496946 612315.141484146 l -310 -2037 384 -2426 -180 -916 -92 -583 -91 -583 -90 -500 -6 -417 " />;

International nomenclature layer:

<svg viewBox=" 0 0 889986.770735918 616008.63322442">

<g id="Coordinary transform" transform="translate(0,616008.63322442) scale(1,-1)" fill="green" fill-opacity="0.2" stroke="black" stroke-width="941.247127475211">

<g id="gd5wok2.pat">

<path AREA="464395826.299377" PERIMETER="87239.64541733" SysID="2" UserID="760" onclick="selectnum(2)" MAPNUM="G50E016016" OLDNUM="G-50-92-¶¡" d="M 846355.890625 615266 l 7696.33324793004 227.219209500123 …… -25172.518763711 -743.000569839962 -528.9375 18444.25 Z" />;

Figure 5: four layers

We can gain the configuration below(some fields can be null): **Figure 6**

Table DOCUMENT		
id	name	info
1	5w	TLIST path ><!ATTLIST path UserID CDATA #IMPLIED PERIMETER CDATA #IMPLIED GD25WOK2 CDATA #IMPLIED PLIED MAPNUM CDATA #IMPLIED OLDNUM CDATA #IMPLIED ><!ATTLIST ellipse SysID CDATA #IMPLIED UserID T rect MINX CDATA #IMPLIED MINY CDATA #IMPLIED MAXX CDATA #IMPLIED MAXY CDATA #IMPLIED >]

table SVGTG											
Did	viewBox	svgid	Tid	transform	fill	fill-opacity	stroke	stroke-width	Gid	graph	EID
1	0 0 889986.770735 918 616008.633224 42		Coordinary transform	translate(0,616008.63322442) scale(1,-1)	mintcream		black	941.24712 7475211	bndcodeok2.pat	path	E1

table E1/E2/E3
d

table E4							
AREA	PERIMETER	SysID	UserID	onclike	MAPNUM	OLDNUM	d

Figure 6: example tables

Every layer has a graphic element table. In table SVGTG "EID" is used to point to the graphic element table. Table E1, E2, E3 have the same field. Field "d" can contain a long string; maybe SQL SERVER ntext type is a better choice.

4.3 storage of SVG data into SVG database

In order to store SVG data into relational database like above, first the whole document should be read. It comes down to the xml document disposal mode. Since xml becomes a recommended language in 1998, there are two modes. The first one is DOM (Document Object Model), but it can't satisfy all kinds of needs of application especially when dealing with a big XML document. Then the second one is SAX (Simple API for XML). It offers a PUSH mode [6] , which is better than DOM. However it parses the xml document in such an extraordinary complex way that it is hard to use. Microsoft .NET Framework has imported a new mode--PULL mode. It's a Forward-only, no buffer-visited-mode which is high efficient and easier to use. It offers the class XMLReader which is used to read and write the whole document outstandingly but not to insert, update or delete some nodes. With the help of ADO.NET, SVG data can be saved into SQL SERVER because SQLCONNECTION serves the fastest connection.

```
//create reader to read svg file "filename"
 XmlTextReader reader=new XmlTextReader(filename);
//overleap the document head to find the first layer
if(reader.MoveToContent()==XmlNodeType.Element && reader.Name=="svg")
{
//read the first layer
    reader.read();
    while(reader!=null)
    {   //read element <svg> and its attributes of viewBox etc.
        for(i=0;i<reader.Attributecount;i++)
        {
            reader.MoveToAttribute(i); attrname[i]=reader.Name; attr[i]=reader.Value;
        }
        reader.MoveToElement();
        //read element <g> and its attributes of transform etc.
        reader.read();
        for(j=0;j<reader.Attributecount;j++,i++)
        {
            reader.MoveToAttribute(j); attrname[i]=reader.Name; attr[i]=reader.Value;
        }
        reader.MoveToElement();
        //put all these attribute into table SVGTG as a record
        ...
        //read graphic element such as path of the first layer and put into DataSet
        depth=reader.Depth+1;countlayer=countlayer+1;DataSet ds=new DataSet();DataRow oRow;
        mycom.CommandText="INSERT INTO E1 (...) VALUES (....)";
        mycom.FillDataSet(ds,"E1");
        while(reader.read() && reader.Depth==depth)
        {
           oRow=ds.Tables["E1"].NewRow();
           for(i=0;i<reader.Attribute;i++)
          {
              reader.MoveToAttribute(i); attrname[i]=reader.Name;
              attr[i]=reader.Value; oRow(attriname)=attr[i];
          }
          ds.Tables["E1"].Rows.Add(oRow);
          reader.MoveToElement();
          depth=reader.Depth;
}
//use the DataSet record to update the first layer table E1
  mycom.Update(ds,"E1");
//deal with other layer...
```

Figure 7: code used to store SVG data into SVG database

4.4 issuance and alternation

In the past, a big deal of spatial data is stored in one SVG document and sent to browser at one time. It has bad effect on the issuance speed. Now in this system when the browser sends out URL, the first sent SVG data is only administrated district layer. Later, when the browser asks for other layers by check box, the WEB server will be back to the SVG data to find out more information. The result is first stored in the DataSet. Then it is written into document in SVG format in order to display in the browser correctly. The interface of the example is in **Figure 8**

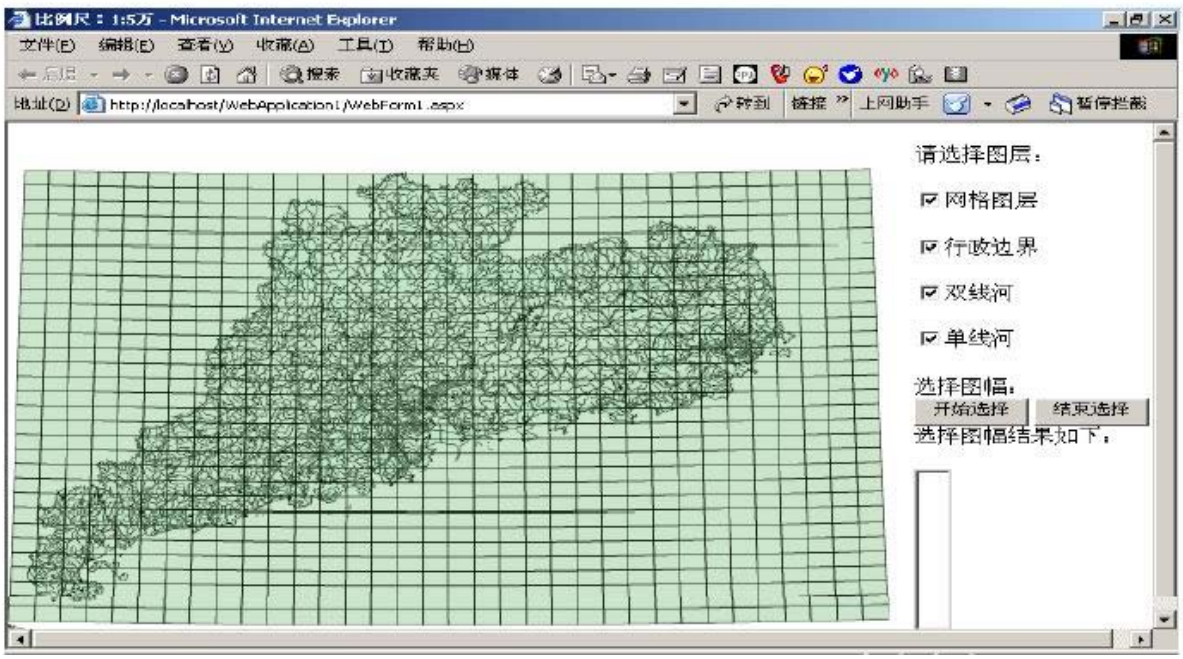


Figure 8: the interface of the example

The code **Figure 9** below stores the result got from SVG database into DataSet provided by ADO.NET. Then it is written into SVG format with field name as attribute name while field value as attribute value.

```
//the result is stored in DataSet dset, in which table[0] is the record queried from table SVGTG,
//table[1] is the graphic layer table that want to display
//write the result into xml file "filename"
XmlTextWriter writer=new XmlTextWriter(filename);
writer.WriteStartDocument();
writer.WriteDocType("svg",...);
//write element <svg viewBox=...>
writer.WriteStartElement("svg");
DataRow dr=dset.Tables[0].Rows[0];
for(i=0;i<2;i++)
      writer.WriteAttributeString(dset.Tables[0]. Columns[0].ColumnName,dr(0));
//write element<g transform=....> alike
...
//write all the graphic element such as path, iRowcount is the total record number in table[1]
//of DataSet dset, the graphic element name such as path is gained from table[0] graph field
for(i=0;i<iRowcount;i++)
 {
      writer.WriteStartElement(dset.Tables[0].Columns["graph"]);
      DataRow dr=dset.Tables[1].Rows[i];
      for(j=0;j<column;j++)
           writer.WriteAttributeString(ds.Tables[1].Columns[j].ColumnName,dr(j));
      writer.WriteEndElement();
  }
writer.WriteEndElement();
writer.WriteEndElement();
```

Figure 9: code used to transform the result into SVG format

At last, the result file will be sent as a new file or appended into the original file with appendChild().

Such kind of manipulation avoids the congestion of sending all the information at a time. As a result, the customers can get the information which they are interested in conveniently. We could associate each layer with administrated district and international nomenclature, and then the customers can lookup useful information in a pyramid way. It can greatly reduce the data quality.

5. Conclusion and prospect

Representing spatial information in SVG format is a developmental domain. This paper aims to deal with the speed problem which companies with large amount of spatial information issuance on Internet. Establishing a SVG database has greatly improved the efficiency of issuance. In fact there exists some malpractice to store data in document. Storing data in database has many advantages. The database schema designed in this paper can satisfy common needs of manipulating elements and their attributes. And it can diminish the time wasted in DOM. For example, there are nearly ten thousand polygon elements in a layer; they're needed to modify the color attribute; it will waste a much longer time (almost several minutes) to finish

this operation through DOM because of its configuration problem. However it can save a lot of time to write a new document through document stream, which has disadvantages of course. It has to waste a lot of time to query in database and rewrite. Medium granularity method can be chosen when it is not needed to manipulate elements etc. Admittedly this system is not perfect, it just realize a few functions. A lot of researches are still needed to be carried over.

Bibliography

[1]

Gong Jianya. Data organization and dealing measures of integral GIS, Wuhan: Press of Wuhan Technical University of Surveying and Mapping, 1993 (in Chinese).

[2]

Guo Renzhong. The classification and configuration of spatial objects, Journal of Wuhan Technical University of Surveying and Mapping, 1994, 19(1): 22~27 (in Chinese).

[3]

Mark Graves. Designing XML Databases, Pearson Education Inc., 2002.

[4]

Ronald Bourret. XML and Databases, last update July, 2004. http://www.rpbourret.com/xml/XMLAndDatabases.htm

[5]

Ronald Bourret. Mapping DTDs to Databases, May 09, 2001. http://www.xml.com/pub/a/2001/05/09/dtdtodbs.html

[6]

Zheng Xiaoping. Visual C#.NET exploitation and practice, Beijing:People's Post and Telecommunications Publishing House, 2001 347~351 (in Chinese)

XHTML rendition created by gcapaper Web Publisher v2.0, © 2001-3 Schema Software Inc.